

Alma Mater Studiorum – Università di Bologna

Bachelor's Degree in Management and Economics

Strategy Optimization via Synthetic Market Generation

Supervisor:
Prof. Giovanni Cardillo

Candidate:
Jacopo Michelacci

Academic Year 2025/2026

Abstract

This thesis studies robust strategy optimization in systematic trading. The main question is how trading strategies can be optimized on past market data while limiting optimization-induced overfitting, defined as the risk that selected parameters fit noise or path-specific patterns in the in-sample period rather than a stable market effect.

The project develops a C++ and Python research framework for backtesting, parameter optimization, trade reconstruction, synthetic market generation, and report generation. The C++ component handles simulation and optimization, while Python is used for plotting and performance analysis.

The empirical analysis compares two optimization workflows. The benchmark uses the standard IS/OOS approach: parameters are selected on the historical in-sample period, defined as the first 70% of the original series, and then tested on the final 30% OOS period. The alternative workflow uses the same OOS test, but changes the optimization input. Instead of optimizing on one historical IS path, it generates multiple synthetic market paths from the IS period and selects parameters using average performance across those paths.

The comparison focuses on the deterioration from optimization performance to OOS performance. A smaller IS/OOS performance gap is interpreted as evidence of lower overfitting. In the experiments, the synthetic optimization method does not show a clear reduction in overfitting compared with the standard IS/OOS benchmark. The synthetic paths are useful as alternative market scenarios, but they do not lead to parameters that generalize better in the tested setting.

Contents

Abstract	i
1 Introduction	1
1.1 The overfitting problem	1
1.2 Rationale	2
1.3 Research question	2
1.4 Approach and contribution	3
2 Methodology	4
2.1 Overview	4
2.2 Data split	4
2.3 Backtesting model	5
2.4 Stop-loss execution	5
2.5 Position and trade accounting	6
2.6 Performance metrics	6
2.7 Measuring overfitting	7
2.8 Optimization protocol	7
2.9 Implementation	8
3 Trading Strategies	9
3.1 Overview	9
3.2 Common strategy structure	9

3.3	Moving-average crossover strategy	10
3.4	Standard-deviation mean-reversion strategy	10
3.5	Role in the thesis	11
4	Synthetic Market Generation	12
4.1	Purpose of synthetic markets	12
4.2	General structure	12
4.3	Geometric Brownian motion baseline	13
4.4	Rolling-volatility GBM	14
4.5	Threshold-constrained rolling-volatility GBM	15
4.6	Synthetic high and low construction	16
5	Optimization	18
5.1	Overview	18
5.2	Moving-average crossover optimization	18
5.2.1	Stop-loss stage	18
5.2.2	Fast/slow length stage	19
5.3	Standard-deviation mean-reversion optimization	21
5.3.1	Stop-loss stage	21
5.3.2	Length and linked threshold stage	23
5.3.3	Independent threshold stage	25
5.4	Selected parameters	27
6	Results	28
6.1	Overview	28
6.2	Moving-average crossover results	28
6.3	Standard-deviation mean-reversion results	29
6.4	Summary	30

7 Conclusion 31

7.1 Summary of findings 31

7.2 Synthetic markets as realistic path approximators 31

7.3 Does synthetic-market optimization reduce overfitting? 31

7.4 Limitations 32

7.5 Future work 33

Code Availability 34

References 35

Chapter 1

Introduction

1.1 The overfitting problem

Overfitting can take different forms. In this thesis, the term refers specifically to optimization-induced overfitting in trading strategy backtests. This is the risk that a strategy appears strong because its parameters were selected to match noise or path-specific patterns in the IS period, rather than because the strategy captures a stable market effect. White (2000) formalizes this data-snooping problem: when many alternatives are tested on the same sample, strong historical performance can arise by chance. Sullivan et al. (1999) apply this problem directly to technical trading rule performance, while Bailey et al. (2017) study the same issue in the specific context of backtest overfitting.

A naive approach to strategy development would be to optimize a strategy on the full available historical dataset and then deploy the best-performing configuration. This is problematic because the same data is used both to choose the parameters and to judge the quality of the strategy. If many parameter combinations are tested, some of them are likely to look good by chance, even if the underlying rule has little predictive value. Such a configuration may produce an impressive backtest, but fail when applied to new market data.

In this setting, overfitting can therefore be understood as a loss of generalization caused by parameter selection. The strategy becomes too adapted to the noise, anomalies, and specific sequence of events observed in the historical sample. As a consequence, the more a strategy is overfitted, the less likely it is that its backtested performance will be repeated in the future.

A common response to this issue is to split the historical data into two parts. The first part, usually called the in-sample period (IS from now on), is used to develop and optimize the strategy. The second part, called the out-of-sample period (OOS from now on), is kept separate and used only after the parameters have been selected. In this thesis, the benchmark approach uses a 70% in-sample and 30% out-of-sample split, where the out-of-sample period corresponds to the most recent part of the original historical series. This reflects the practical objective of testing whether a strategy optimized

on the past would have remained effective on later unseen data.

1.2 Rationale

Given the overfitting problem discussed above, the standard practice is to separate optimization and evaluation through an IS/OOS split. The strategy is optimized on the IS period, the parameters are frozen, and the final evaluation is performed on the OOS period. This is useful because it creates a cleaner test of whether the optimized parameters generalize to unseen data.

This thesis proposes an alternative optimization input. Instead of optimizing only on the single historical IS path, the IS period is used to generate several synthetic market paths. The strategy is then optimized using average performance across those paths. The purpose is to test whether this can reduce the loss of generalization created by overfitting.

Bailey et al. (2017) address a related but different part of the problem. Their approach estimates the probability that a selected backtest is overfit after many alternatives have been tested. This thesis does not try to estimate that probability directly. Instead, it tests whether changing the optimization input can lead to parameter choices with smaller IS/OOS deterioration.

This idea is not usually the first approach in strategy optimization. A common objection is that synthetic random walks may add too much noise and remove too much of the original market structure. In that case, the optimizer would no longer learn from market behaviour, but from artificial randomness. This thesis addresses that concern by using synthetic paths that retain randomness while remaining linked to the original market path by construction.

1.3 Research question

The central question of this thesis is whether a synthetic-market-based optimization procedure can improve robustness compared with the standard historical in-sample optimization approach. Both methods use the same general optimization logic, but they differ in the data on which the optimization is performed.

In the benchmark case, the strategy is optimized directly on the historical in-sample period. In the synthetic case, the in-sample period is first used to generate multiple synthetic market paths. The strategy is then optimized across these generated paths, and parameter selection is based on average results rather than on one realized historical path. The selected parameters from both approaches are finally tested on the same real out-of-sample period. This makes the comparison focused: the objective is not to prove that a strategy is profitable, but to evaluate which optimization procedure loses less performance when moving from optimization to OOS testing.

1.4 Approach and contribution

The research question requires both a clear testing procedure and code that can run many strategy tests efficiently. For this reason, I developed a custom research framework composed of a C++ backtesting and optimization engine connected to Python reporting and analysis tools. The C++ component handles the computationally intensive parts of the workflow, including data processing, strategy simulation, trade reconstruction, and grid-based optimization. Python is used for report generation and plotting.

The framework is built to compare the proposed method against a clear benchmark. The benchmark is the standard historical IS/OOS workflow: parameters are selected on the real IS period and evaluated on the real OOS period. The proposed method keeps the same OOS evaluation, but changes the optimization input: parameters are selected using the mean result across synthetic IS paths. In both cases, the same strategy, data split, performance metrics, and OOS test period are used. The main comparison is therefore the IS-OOS performance gap produced by each method.

The practical relevance of this comparison is capital allocation. An asset manager does not only need a strategy that performs well in a backtest; they need evidence that the selected parameters are not mainly the result of fitting noise in one historical sample. A method that reduces the IS-OOS performance gap would make strategy selection more reliable before capital is allocated. Conversely, if the synthetic workflow does not reduce this gap, this is also useful information, because it shows that the additional computational cost is not justified by better OOS robustness in the tested setting.

Chapter 2

Methodology

2.1 Overview

The methodology is designed to compare two ways of optimizing trading strategies while keeping the final test as clean as possible. The first method is the standard and also our benchmark: optimize the strategy on past historical data and then test it on later unseen data. The second method uses synthetic markets: the synthetic paths are generated from the same IS period, the strategy is optimized across those paths, and the selected parameters are then tested on the same real OOS period.

The important point is that both methods are evaluated on the same final historical window. This makes the comparison focused on the optimization process rather than on differences in the test data. The synthetic approach is considered useful only if the parameters selected from synthetic paths transfer better to the real OOS period.

2.2 Data split

The historical dataset is split chronologically. The first part of the series is used as the IS period, while the final part is held out as the OOS period. In the experiments, the standard split is 70% IS and 30% OOS.

The IS data is used for strategy optimization and, in the synthetic approach, for generating synthetic market paths. The OOS data is not used during parameter selection. It is only used after the parameters have been chosen, in order to evaluate how the strategy behaves on unseen historical data.

This chronological split is important because financial data is time-ordered. Randomly shuffling observations would not represent the real situation faced by a trader, where parameters are chosen using past information and then applied to future market data.

All comparisons are performed on the same asset and the same OOS period.

2.3 Backtesting model

The backtester operates on OHLCV bars. Each bar contains open, high, low, close, volume, and timestamp information. Strategy logic is evaluated after a bar is available, and normal market orders generated by the strategy are filled at the open of the following bar.

This execution rule is used to avoid lookahead bias. A strategy is not allowed to observe a bar and then trade at a price from that same bar. For example, if a signal is generated after observing bar t , the earliest normal execution price is the open of bar $t + 1$.

The exception is stop-loss execution. Stop losses are treated as protective orders that are already active once a position has been opened. Therefore, if the high or low of a bar crosses the stop level, the stop is considered triggered inside that bar and is filled at the stop price. This is different from a new strategy signal, because the stop level was known before the bar was completed.

The equity curve is marked once per bar using the close price:

$$\text{Equity}_t = \text{Cash}_t + \text{Open Position}_t \cdot \text{Close}_t.$$

Therefore, drawdowns computed from the equity curve are close-to-close drawdowns. Intrabar high and low prices are not used to mark the equity curve, but they are used for stop-loss execution.

2.4 Stop-loss execution

Stop losses are treated as protective orders attached to open positions. This reflects the way a live system would typically work: after an entry is filled, a stop order can be placed at the broker and remain active while the position is open.

Because the available data is OHLCV bar data, the exact intrabar path is unknown. The backtester therefore uses the following convention: a long stop loss is considered triggered if the bar low is less than or equal to the stop price, while a short stop loss is considered triggered if the bar high is greater than or equal to the stop price. When triggered, the stop is filled at the stop price.

For a percentage stop loss, the stop level is computed from the entry price. For a long position:

$$\text{Stop Price} = \text{Entry Price} \cdot (1 - \text{Stop Percentage}).$$

For a short position:

$$\text{Stop Price} = \text{Entry Price} \cdot (1 + \text{Stop Percentage}).$$

The framework also supports a notional stop distance. In that case, the stop is placed at a fixed price distance from the entry. If both a percentage stop and a notional stop are provided, the backtester uses the tighter stop.

This stop-loss model is an approximation. It assumes that a stop is filled at the stop price once the bar high or low crosses it, and it does not model the exact intrabar sequence of prices. Transaction costs, including fees or an assumed slippage component, can be included by specifying a cost in basis points at the start of the backtest run.

2.5 Position and trade accounting

The backtester tracks both the net open position and the individual open lots. The net position represents the total current exposure. The open-lot log stores the individual entries that are still open.

This distinction is useful when stacking is allowed. If two buy orders are filled separately, they increase the same net long exposure, but they remain two distinct open lots. This makes it possible to track independent entry prices and independent stop losses.

When an order reduces an existing position, open lots are closed using FIFO. For example, if two long lots of quantity 1 are open and a sell order of quantity 1.2 is filled, the first lot is closed completely and the second lot is reduced to quantity 0.8.

This accounting also affects trade reconstruction. A trade is considered closed when an open lot, or part of an open lot, is closed by an opposite-side order or by a stop loss.

2.6 Performance metrics

The framework computes several metrics to evaluate each backtest:

- **Net profit:** final equity minus initial capital.
- **Average trade:** average profit or loss per completed trade.
- **Sharpe ratio:** annualized mean return divided by return volatility.
- **Maximum drawdown:** largest percentage decline from a previous equity peak.
- **Number of trades:** total number of completed trades.
- **Trades per year:** number of completed trades normalized by the length of the test period.

These metrics are used both during optimization and during final OOS evaluation. The purpose is not to rely on a single number only, but to evaluate a strategy on risk-adjusted return and frequency metrics.

2.7 Measuring overfitting

Overfitting is assessed by comparing optimization performance with OOS performance. For each method, the selected parameters have one performance value during optimization and one performance value on the real OOS period. If performance is much worse OOS, the optimized parameters likely captured too much of the specific optimization sample.

For a metric M where higher values are better, the performance difference is defined as:

$$\text{Diff} = M_{OOS} - M_{IS}.$$

With this convention, a more negative value means that more performance was lost when moving from IS optimization to OOS testing. In this thesis, the synthetic optimization method is considered less overfit if it produces a smaller deterioration than the historical benchmark on the same strategy and OOS period.

For metrics where lower values are better, such as max DD, the interpretation is reversed: the relevant deterioration is an increase from IS drawdown to OOS drawdown.

2.8 Optimization protocol

Optimization is performed using grid search. A parameter grid is defined, the strategy is backtested for each parameter value, and the resulting metrics are recorded. The optimizer is implemented in C++ so that repeated backtests can be run efficiently.

Parameter selection is not intended to be a blind maximization of one metric. A parameter value that produces the best result IS may simply be fitted to that specific historical window. For this reason, the optimization plots are inspected with robustness in mind. The goal is to prefer parameter regions that show low variation across nearby values, rather than isolated peaks.

In the historical benchmark, the grid search is run directly on the historical IS period. In the synthetic approach, the same strategy and parameter grid are evaluated on multiple synthetic paths generated from the IS data. The optimization result is then based on mean performance across synthetic paths.

After parameter selection, the chosen parameters are frozen. Both the historically optimized version and the synthetically optimized version are then tested on the same real OOS period.

2.9 Implementation

The framework is implemented using C++ and Python. C++ is used for data loading, strategy execution, backtesting, optimization, and synthetic market generation. Python is used for plotting and report generation.

The C++ code was written with runtime efficiency in mind. The backtester keeps the main simulation loop in C++, avoids writing full report files during optimization runs, and reuses already-loaded data when evaluating multiple parameter values. The indicator interface uses the curiously recurring template pattern (CRTP), avoiding virtual dispatch in repeated indicator updates. Strategy objects are still accessed through a simple polymorphic interface, which keeps the backtesting interface flexible.

All experiments are run with the C++ code compiled in release mode, enabling compiler optimizations. The code is not parallelized, so each optimization run is executed sequentially. On the local test machine, a MacBook Air with an Apple M1 processor and 8 CPU cores, one optimization backtest run (daily data, 2000-2026) takes approximately 1.2–1.8 ms in the tested configurations.

Chapter 3

Trading Strategies

3.1 Overview

The empirical part of the thesis uses rule-based trading strategies. The goal is not to prove the profitability of a trading strategy, but to compare how different optimization procedures affect the same strategy when tested OOS. For this reason, the strategies are intentionally simple and interpretable.

Two strategy families are implemented in the framework: a moving-average crossover strategy and a standard-deviation mean-reversion strategy. They represent two common types of systematic trading logic. The first is trend-following, while the second is mean reverting.

3.2 Common strategy structure

All strategies follow the same execution interface. At each bar, the strategy receives the current market observation and the current portfolio context. It can then produce one or more order events. The backtester is responsible for executing those orders according to the execution rules described in the methodology chapter.

Strategies can be configured to allow or disable long trades, short trades, and stacking. Stacking means that a strategy is allowed to add to an existing position in the same direction. If stacking is disabled, the strategy cannot increase an already open position on the same side.

In the experiments, a fixed position size is used. This keeps the comparison focused on the optimization procedure and the trading rule, rather than on the effects of dynamic position sizing or compounding. Although fixed-fractional position sizing could represent the effect of the two optimization methods more realistically, it is not the sizing method used for the thesis results. The framework supports dynamic sizing based on current equity values, but it is left outside the empirical comparison.

Both strategies can attach a hard stop loss to newly opened positions. The stop can be defined either

as a percentage move from the entry price or as a fixed notional price distance. Stop-loss execution is handled by the backtester at the open-lot level.

3.3 Moving-average crossover strategy

The moving-average crossover strategy is a trend-following rule. Moving-average rules are a standard example of technical trading rules in empirical finance. Brock et al. (1992) test moving-average and trading-range-break rules on the Dow Jones Industrial Average and show that such simple rules have been widely used as empirical tests of market predictability. The strategy used here compares a fast moving average with a slow moving average. The fast average reacts more quickly to recent price changes, while the slow average represents a smoother estimate of the price trend.

Let $MA_f(t)$ be the fast moving average at time t , and $MA_s(t)$ be the slow moving average. A long signal is generated when the fast moving average crosses above the slow moving average:

$$MA_f(t) \geq MA_s(t) \quad \text{and} \quad MA_f(t-1) < MA_s(t-1).$$

A short signal is generated when the fast moving average crosses below the slow moving average:

$$MA_f(t) \leq MA_s(t) \quad \text{and} \quad MA_f(t-1) > MA_s(t-1).$$

The main parameters of this strategy are the fast moving-average length, the slow moving-average length, and the stop-loss value.

3.4 Standard-deviation mean-reversion strategy

The standard-deviation mean-reversion strategy is a contrarian rule. Instead of following the direction of recent movement, it looks for unusually large one-bar moves and assumes that part of the move may revert.

At each bar, the strategy computes the last price move:

$$\Delta P_t = P_t - P_{t-1}.$$

It then computes a rolling standard deviation of recent price moves. The latest move is standardized by dividing it by this rolling standard deviation:

$$Z_t = \frac{\Delta P_t}{\sigma_t}.$$

where σ_t is the rolling standard deviation over the selected lookback window.

A long signal is generated when the standardized move is below a lower threshold:

$$Z_t \leq \theta_L.$$

A short signal is generated when the standardized move is above an upper threshold:

$$Z_t \geq \theta_U.$$

The intuition is that a large negative move may be followed by a rebound, while a large positive move may be followed by a pullback. The main parameters of this strategy are the standard-deviation lookback length, the lower threshold, the upper threshold, and the stop-loss value.

3.5 Role in the thesis

The two strategies are used as test cases for the optimization methodology. They are deliberately simple enough to make their behaviour understandable, but different enough to test the optimization procedure on both trend-following and mean-reversion logic.

The exact parameter grids and selected parameter values are discussed in the optimization and results chapters.

Chapter 4

Synthetic Market Generation

4.1 Purpose of synthetic markets

The central problem of this thesis is not only whether a strategy performs well on one historical sample, but whether the chosen parameters are robust. A single historical price series gives only one realised path of the market. If the optimization is based only on that path, the selected parameters may fit details that are specific to that sample and may not repeat in the future.

Synthetic markets are used to create alternative price paths that preserve some basic statistical properties of the original data, while still producing different possible market evolutions. Politis and Romano (1994) introduce the stationary bootstrap as a way to resample dependent time series. The synthetic paths used here are not a direct implementation of that bootstrap, but they serve a similar purpose: creating alternative market paths without treating the observations as independent.

In the thesis workflow, synthetic paths are generated from the historical IS data. The strategy is then optimized across those paths, using average performance across simulations instead of performance on one single historical IS sample. The final comparison is still made on the real OOS period, so the synthetic data is used only during the optimization phase.

4.2 General structure

Each synthetic path keeps the same timestamps and number of observations as the original data. The generator copies the structure of the original OHLCV series and then replaces the price values with simulated prices. Volume was not modeled nor considered in the synthetic generation as the strategies chosen operate without it.

All methods start from the first observed close price. For each following bar, the previous synthetic close is used as the next synthetic open. A synthetic close is then generated, and the high and low

are constructed around the synthetic open and close. This keeps the generated series in a standard OHLCV format.

The generator currently implements three methods:

- a basic geometric Brownian motion baseline;
- a rolling-volatility geometric Brownian motion;
- a rolling-volatility geometric Brownian motion with threshold constraints.

In the synthetic path comparison shown in this chapter, each generator produces 50 paths. The rolling methods use a 30-bar window for volatility estimation. The threshold-constrained version uses the same 30-bar volatility window and a threshold multiplier of $k = 10$. The first two generators are included for comparison; the empirical optimization and results use only the threshold-constrained version.

In the figures, the red line represents the original historical market path, while the gray lines represent the generated synthetic paths.

4.3 Geometric Brownian motion baseline

The first method is a basic geometric Brownian motion (GBM). It is included as a simple benchmark because it is easy to interpret and widely used as a first approximation for price simulation.

In continuous time, GBM is usually written as:

$$dP_t = \mu P_t dt + \sigma P_t dW_t,$$

where P_t is the price, μ is the drift, σ is the volatility, and dW_t is the Brownian motion shock. The exact discrete-time solution over one time step is:

$$P_t = P_{t-1} \exp \left((\mu - 0.5\sigma^2)\Delta t + \sigma\sqrt{\Delta t}z_t \right), \quad z_t \sim N(0, 1).$$

In the implementation, one time step corresponds to one bar of the input data, so $\Delta t = 1$.

The method first computes historical close-to-close log returns:

$$r_t = \log \left(\frac{P_t}{P_{t-1}} \right).$$

The mean and standard deviation of these returns are used as the per-bar drift and volatility estimates. The next synthetic close is therefore generated as:

$$P_t^{syn} = P_{t-1}^{syn} \exp \left((\mu - 0.5\sigma^2) + \sigma z_t \right),$$

where z_t is drawn from a standard normal distribution. The time step is one bar, so there is no explicit dt term in the implementation. The estimates are already expressed in the frequency of the input data.

This baseline is intentionally simple. Its main limitation is that it uses one global volatility estimate for the whole sample. Real markets usually show periods of higher and lower volatility, so a constant volatility model can produce paths that are too smooth in turbulent periods and too volatile in calm periods.

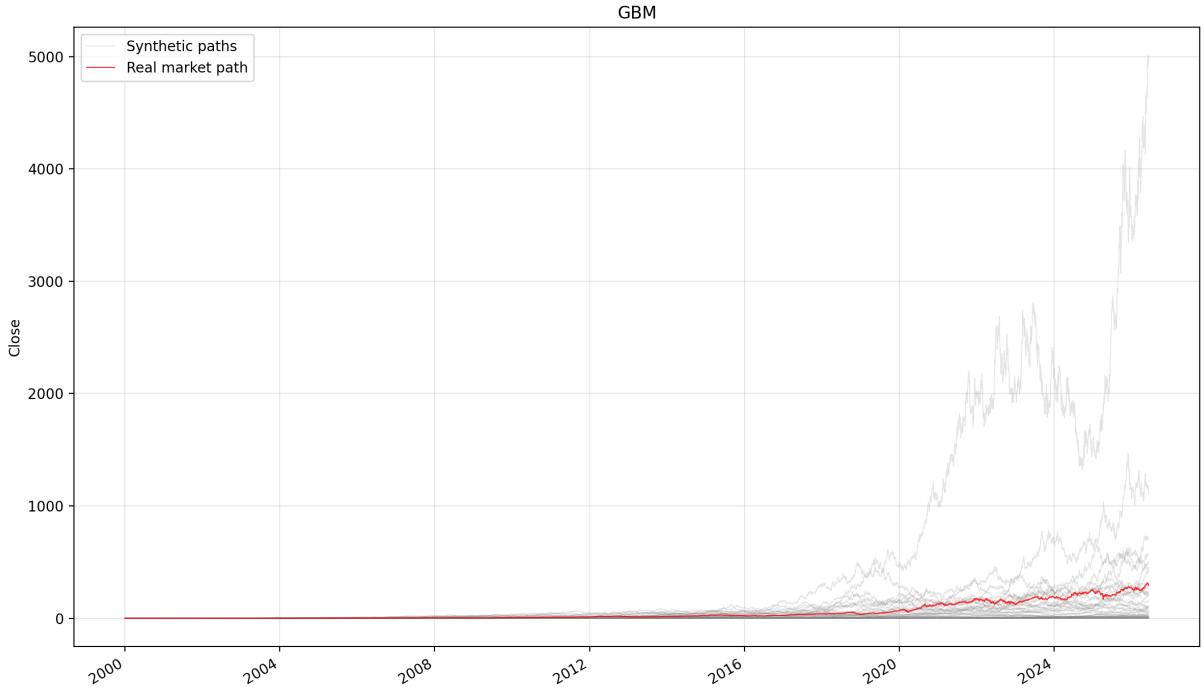


Figure 4.1: Synthetic paths generated with the basic GBM method.

The basic GBM paths move far away from the original series. This makes them less useful for optimization, because they no longer resemble the IS market environment closely enough.

4.4 Rolling-volatility GBM

The second method keeps the same GBM structure but replaces the global volatility estimate with a rolling one. Instead of using one standard deviation for the entire sample, the volatility used at each bar is estimated from the most recent returns in a rolling window. In the experiments shown here, this window is 30 bars.

The synthetic close is still generated from:

$$P_t^{syn} = P_{t-1}^{syn} \exp \left((\mu - 0.5\sigma_t^2) + \sigma_t z_t \right),$$

but σ_t now changes over time. This makes the synthetic paths closer to the structure of the original market, because volatility clusters are partly preserved. If the original IS data contains a volatile region, the generated paths around that region also tend to have larger moves.

The drift μ is kept as a global per-bar estimate. This keeps the method simple and avoids adding too many degrees of freedom. The main feature of this generator is the changing volatility, not a changing expected return.

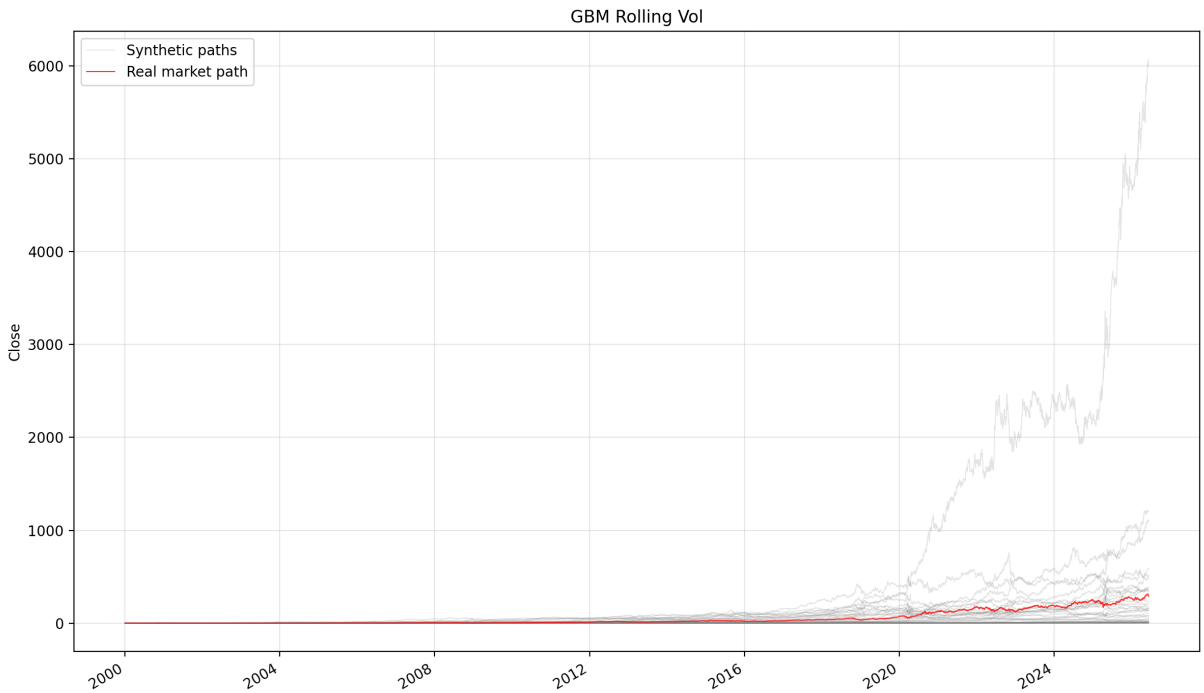


Figure 4.2: Synthetic paths generated with the rolling-volatility GBM method.

The rolling-volatility version is more realistic because volatility changes through time. It is not always more volatile than the basic GBM: it is calmer in low-volatility regions and more volatile in turbulent ones. However, the paths can still drift too far from the original series.

4.5 Threshold-constrained rolling-volatility GBM

The third method adds a constraint to the rolling-volatility generator. At each bar, the generator builds a local price band around the original historical price level. The width of the band is based on the rolling volatility estimate:

$$P_{t-1}^{orig} \exp(-k\sigma_t) \leq P_t^{syn} \leq P_{t-1}^{orig} \exp(k\sigma_t),$$

where k is the threshold multiplier.

If a sampled synthetic close falls outside this band, the value is rejected and sampled again. If the generator cannot find an accepted value after a fixed number of attempts, the value is clipped to the nearest boundary. This prevents extreme synthetic paths from drifting too far away from the original series while still allowing random variation around it.

This method is useful for robustness testing because it creates alternative paths that remain locally anchored to the original market regime. It is less free than the pure GBM generators, but it can produce synthetic data that is easier to compare with the historical IS sample.

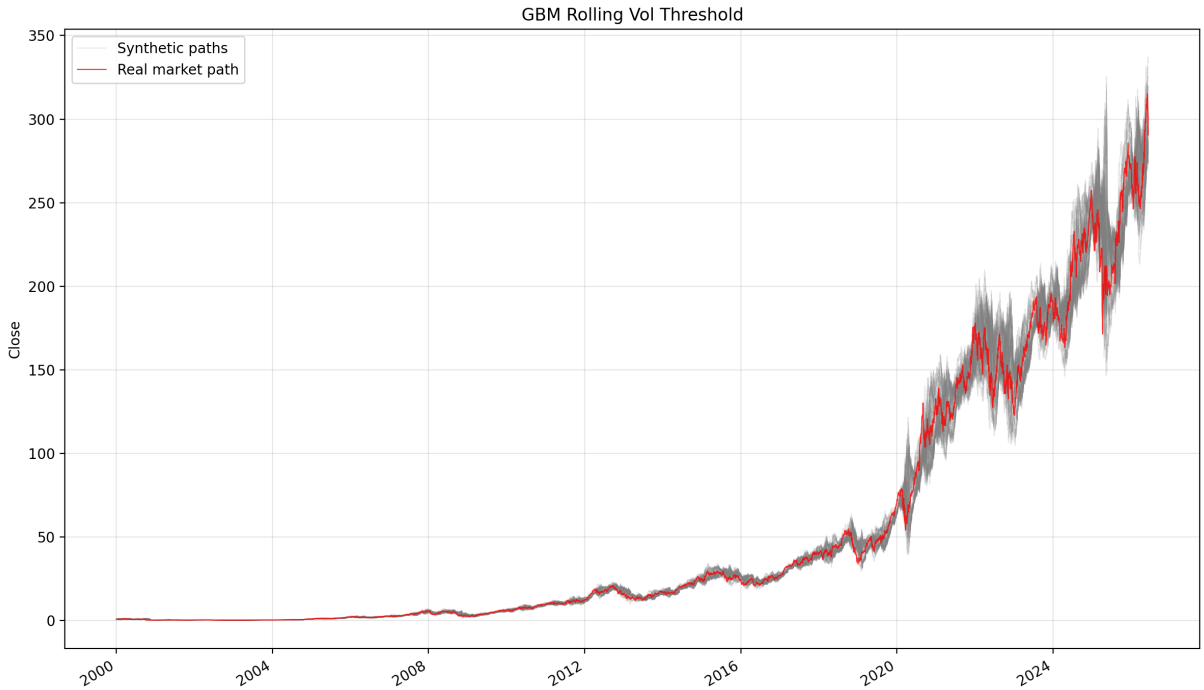


Figure 4.3: Synthetic paths generated with the threshold-constrained rolling-volatility GBM method.

The threshold-constrained version is the most useful for this thesis. It keeps randomness, but remains linked to the original market by construction. The generated paths are therefore alternative versions of the same broad market environment, rather than unrelated price histories. For this reason, only this third generator is used in the optimization and results chapters.

4.6 Synthetic high and low construction

The strategy backtester works with OHLCV bars, not only close prices. For this reason, the generator also creates synthetic highs and lows. The method estimates the historical size of the upper and lower

bar extensions. For each bar, the upper extension is measured relative to the larger of the open and close, while the lower extension is measured relative to the smaller of the open and close. As below:

$$\text{High}_t \geq \max(\text{Open}_t, \text{Close}_t), \quad \text{Low}_t \leq \min(\text{Open}_t, \text{Close}_t).$$

For the basic GBM, the generator uses global mean and standard deviation estimates for these extensions. For the rolling-volatility methods, the generator uses rolling estimates. This makes the intrabar range larger during high-volatility regions and smaller during low-volatility regions.

Chapter 5

Optimization

5.1 Overview

Each strategy is optimized in stages. The stop-loss percentage is usually optimized first. All parameters that are not optimized are kept fixed, either at their resting values or at values selected in previous stages. Parameter selection is based on robust grid regions rather than isolated metric peaks, because selecting the single best point after testing many alternatives is exactly the type of data-snooping problem discussed by White (2000).

The historical grids are evaluated directly on the real historical IS period. The synthetic grids are evaluated on 50 synthetic paths generated from the same IS period, using the threshold-constrained rolling-volatility GBM generator. The value reported at each synthetic grid point is the mean performance across those paths. In every optimization plot, green indicates better values for the metric shown and red indicates worse values.

The metrics reported throughout the chapter are net profit, average trade, Sharpe ratio, max DD, and trades per year.

5.2 Moving-average crossover optimization

5.2.1 Stop-loss stage

The stop-loss percentage was optimized first, holding the rest of the parameters at their resting values (`fast_len = 50`, `slow_len = 100`). Figure 5.1 compares the historical-IS sweep with the synthetic-path mean.

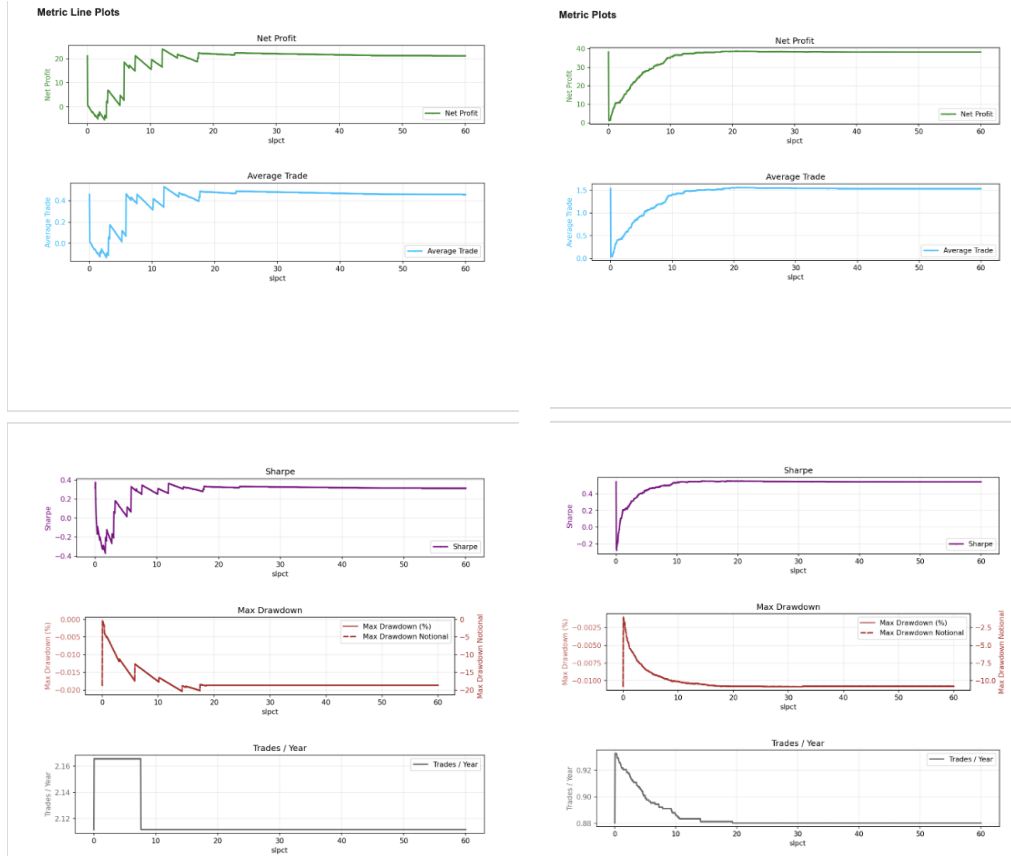


Figure 5.1: Moving-average crossover: stop-loss optimization. Left: historical IS. Right: synthetic mean.

The synthetic plot has a similar shape to the historical one, but it is smoother. This is expected, since every point is averaged across 50 paths instead of being measured on one path only.

The stop-loss was left disabled ($slpct = -1$) for both the benchmark and the synthetic configuration because short stop-loss values produced unstable results.

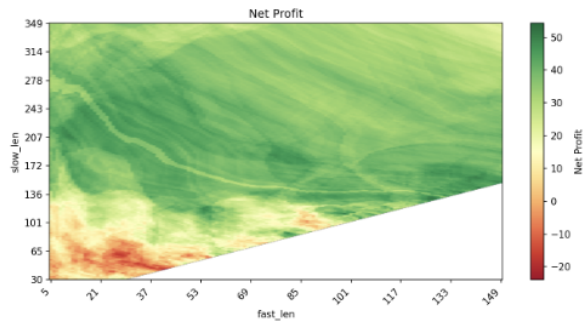
5.2.2 Fast/slow length stage

With the stop-loss disabled, the fast and slow moving-average lengths were optimized jointly. Figures 5.2–5.3 compare the historical-IS grid with the synthetic-path mean.

Historical IS

Synthetic mean

Metric Plots



Metric Plots

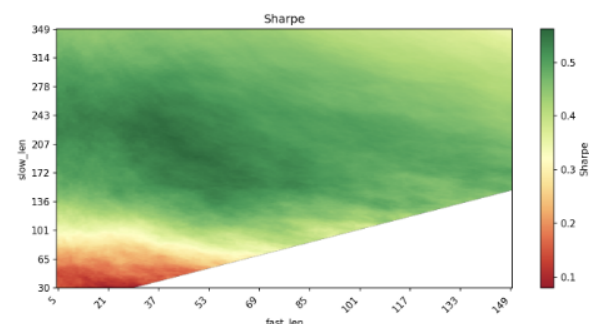
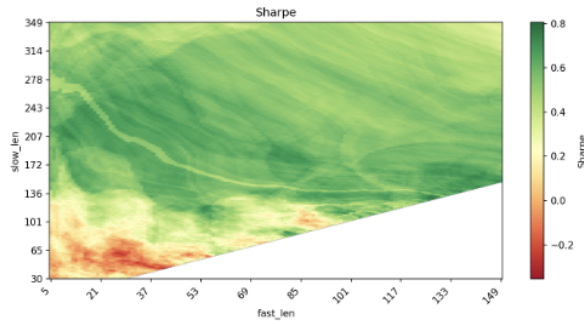
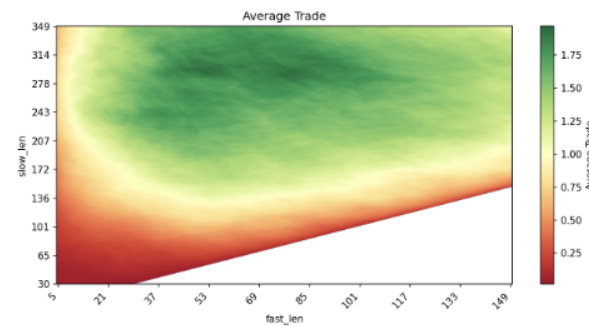
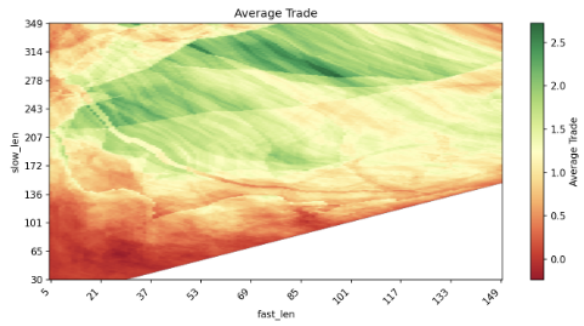
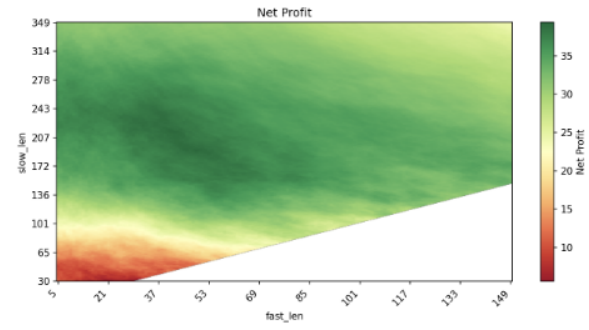


Figure 5.2: Moving-average crossover: first metric panel over the fast/slow length grid.

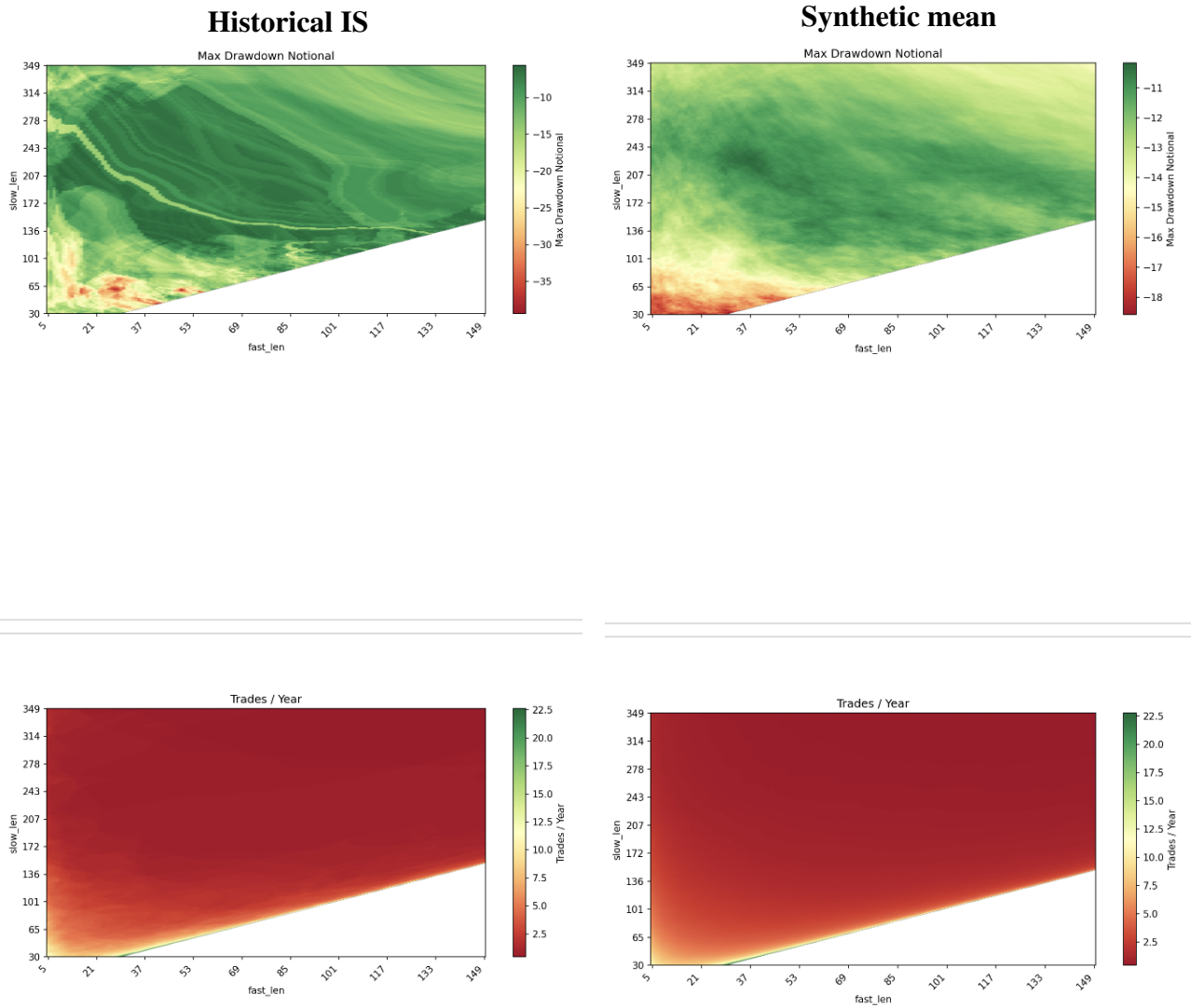


Figure 5.3: Moving-average crossover: second metric panel over the fast/slow length grid.

Both grids show the same main pattern. Results are weak when the fast and slow lengths are close, because the strategy trades too often on small price moves. Results improve when the slow length is larger, creating a broad stable area near $slow_len = 200$. From this region, I chose $fast_len = 50$, $slow_len = 200$ for the benchmark configuration. For the synthetic configuration, I kept the same slow length but chose a shorter fast length: $fast_len = 35$, $slow_len = 200$.

5.3 Standard-deviation mean-reversion optimization

5.3.1 Stop-loss stage

As with the crossover strategy, the stop-loss percentage was optimized first, holding the standard-deviation length and thresholds at their resting values ($std_len = 15$, $lower_std_thresh = -1.5$, $upper_std_thresh = 1.5$). Figure 5.4 sweeps the stop-loss percentage from 0% to 60% on the real historical IS period.

Metric Plots

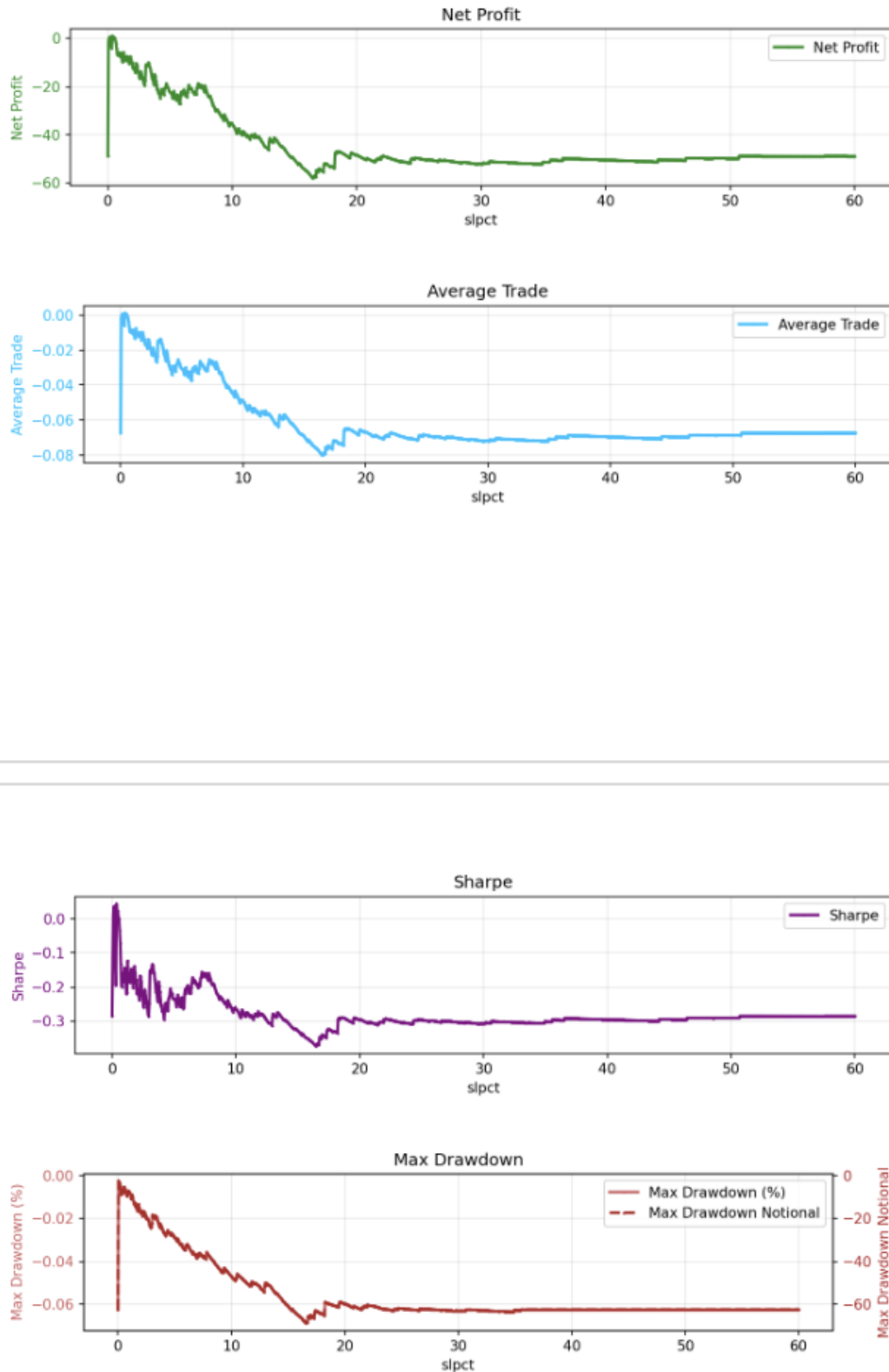


Figure 5.4: Std MRev: metrics as a function of the stop-loss percentage, historical IS.

For this strategy, the stop-loss curve has a clearer structure. Net profit and Sharpe improve for small stop-loss values, around 1%–3%, then deteriorate and eventually flatten once the stop is so wide that it rarely triggers. That flat region is stable, but it is not useful: a mean-reversion trade should be cut when

the move keeps going against it. For this reason, the benchmark stop-loss was selected from the early profitable region, giving $\text{slpct} = 2.5\%$. This is a risk-control choice, not a pure maximum-metric choice.

The same sweep was repeated on synthetic paths, holding the same resting values. Figures 5.5–5.5 report the synthetic-path mean.

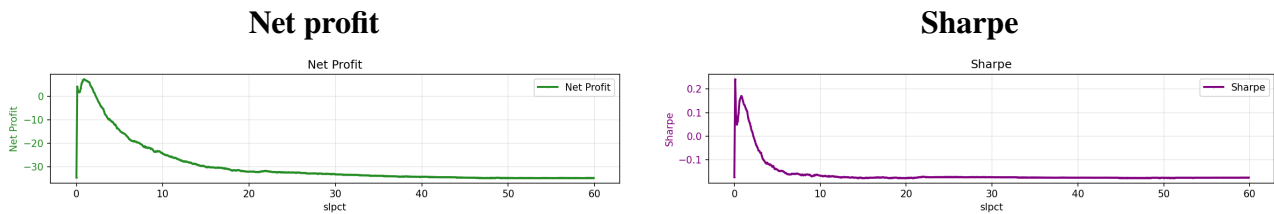


Figure 5.5: Std MRev: stop-loss optimization, mean over synthetic paths.

Averaging over synthetic paths removes most of the noise. The curve rises near the 1% area and then decays toward the same inactive-stop plateau. The best single point, $\text{slpct} = 0.1\%$, is too isolated to use directly. Using the same risk-control logic as above, the synthetic configuration uses $\text{slpct} = 2.0\%$.

5.3.2 Length and linked threshold stage

With the stop-loss fixed, I optimized the standard-deviation length together with one threshold level. To avoid optimizing the length alone, but also avoid a three-parameter grid, the upper and lower thresholds were linked symmetrically: one positive threshold for short entries and one negative threshold of the same size for long entries. Figure 5.6 compares the main historical-IS grid with the synthetic-path mean. Figure 5.7 shows the trades/year surface for the historical-IS grid, since it helps explain how the optimization process was interpreted. The synthetic version was very similar, so it is not repeated here.

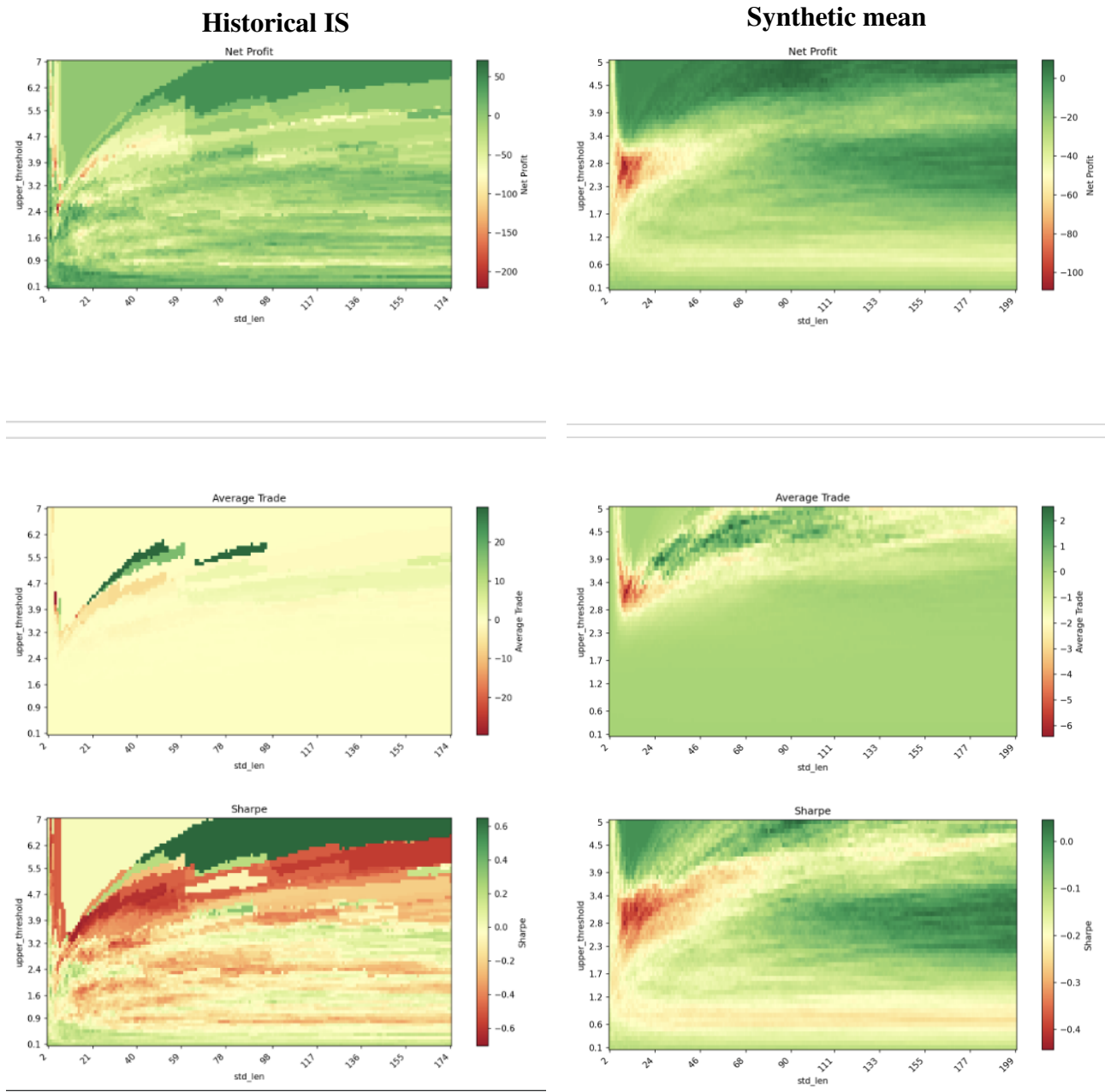


Figure 5.6: Std MRev: first metric panel over the length/symmetric-threshold grid.

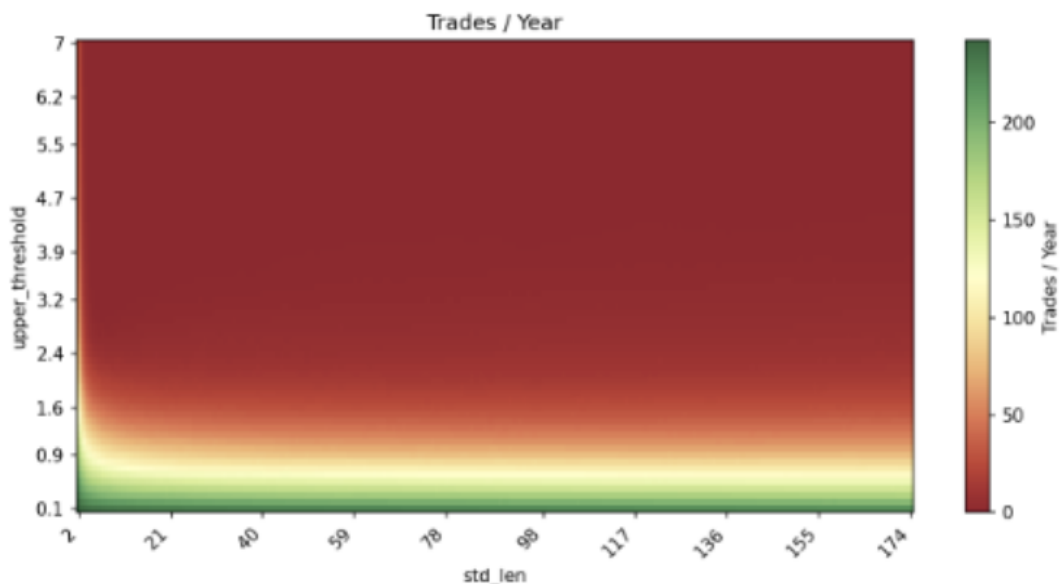


Figure 5.7: Std MRev: trades per year over the length/symmetric-threshold grid, historical IS.

Very short standard-deviation lengths perform poorly because they react to almost every local move. In the historical grid, performance becomes usable around 20–40 bars, so the benchmark configuration uses $std_len = 20$ and a narrow symmetric threshold as the starting point. In the synthetic grid, the stable area appears much later, around 100–150 bars, with a wider threshold. The synthetic configuration therefore uses $std_len = 150$ and starts from a wider threshold. The symmetric threshold is only an intermediate step; the final stage separates the upper and lower thresholds.

5.3.3 Independent threshold stage

Finally, the upper and lower thresholds were optimized independently, holding the standard-deviation length and stop-loss at the values selected in the previous stages. Figure 5.8 compares the main historical-IS grid at $std_len = 20$ with the synthetic-path mean at $std_len = 150$. Figure 5.9 shows the trades/year surface to make the optimization process easier to read. The benchmark and synthetic trades/year plots were very similar, so only this one is reported.

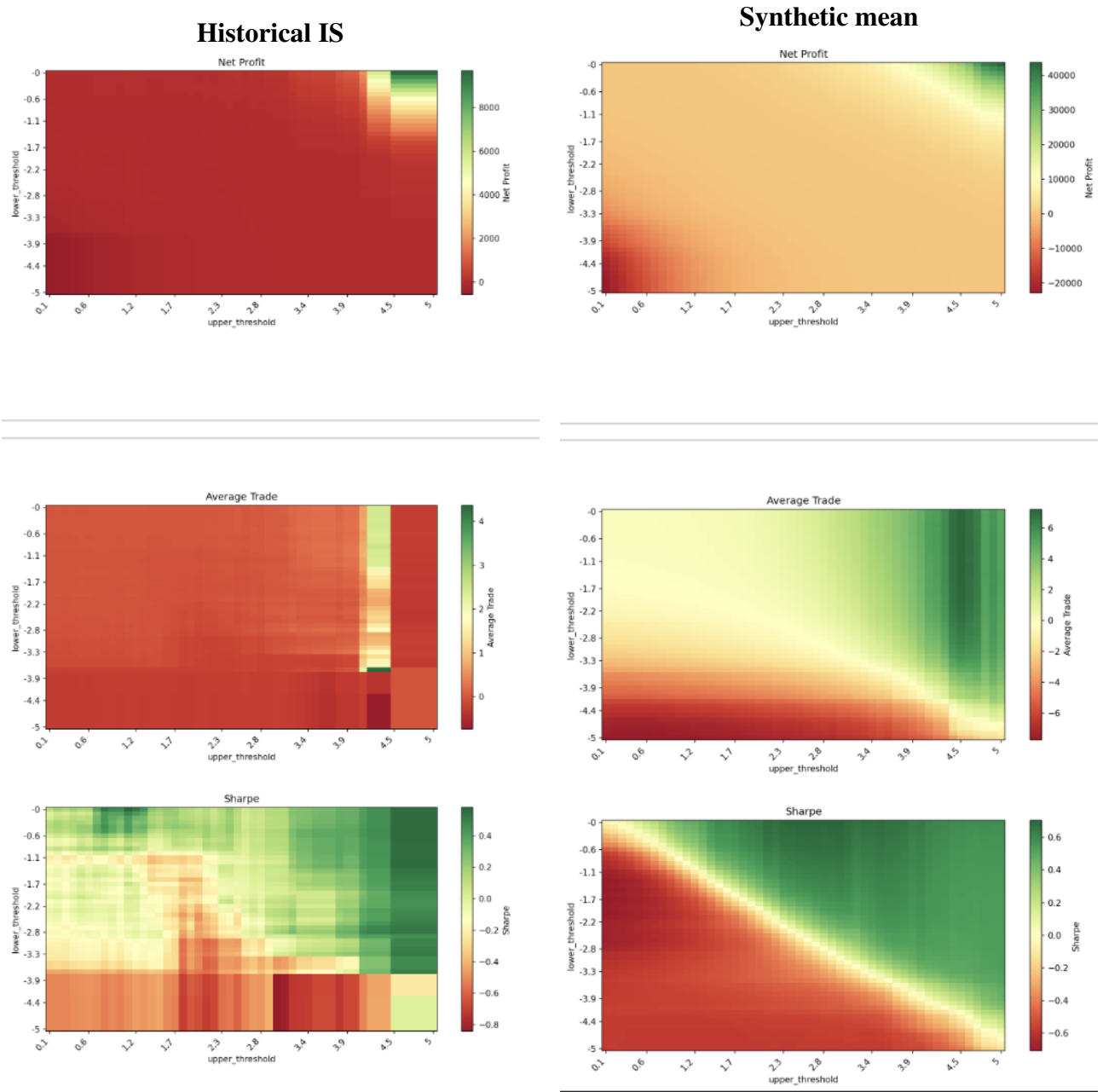


Figure 5.8: Std MRev: first metric panel over the independent upper/lower threshold grid.

Figure 5.8 is one of the clearest examples of why the synthetic-path mean is useful. In particular, the Sharpe surface is much smoother on the synthetic side. Averaging across paths appears to cancel part of the single-path noise and makes the stable region easier to see, although the two panels come from the parameter settings used in each workflow.

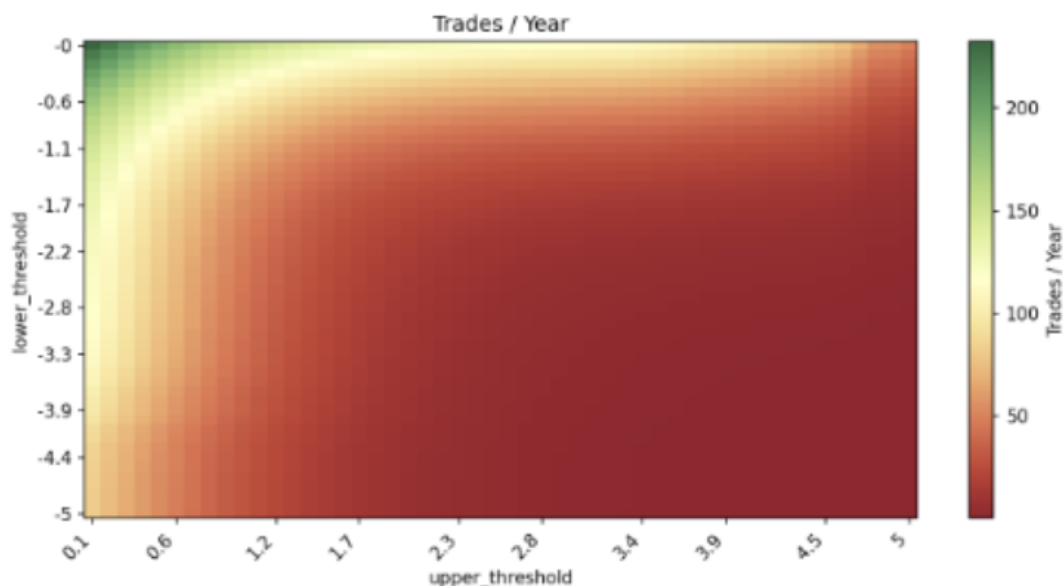


Figure 5.9: Std MRev: trades per year over the independent upper/lower threshold grid.

The independent-threshold grids show that the symmetric rule is not ideal. Performance improves when the lower threshold is kept close to zero and the upper threshold is wider. The final benchmark thresholds are $upper_std_thresh = 1.0$, $lower_std_thresh = -0.35$. The final synthetic thresholds are $upper_std_thresh = 3.0$, $lower_std_thresh = -0.30$. In both cases, the strategy becomes more sensitive to downside moves than upside moves.

5.4 Selected parameters

Table 5.1 summarizes the selected parameters. These configurations are fixed and then evaluated on the real OOS period in the following chapter.

Strategy	Method	Parameters	slpct	Other
MA Cross	Benchmark	fast=50, slow=200	disabled	–
MA Cross	Synthetic	fast=35, slow=200	disabled	–
Std MRev	Benchmark	std_len=20	2.5%	upper=1.0, lower=-0.35
Std MRev	Synthetic	std_len=150	2.0%	upper=3.0, lower=-0.30

Table 5.1: Final selected parameters per strategy and optimization method.

Both methods use the same optimization process: parameters are chosen from robust grid regions rather than isolated peaks. The difference is the optimization data: the benchmark uses the real historical IS period, while the synthetic method uses the mean result across synthetic IS paths. The real test is the next step: comparing IS-to-OOS deterioration and checking whether the synthetic method reduces it.

Chapter 6

Results

6.1 Overview

This chapter reports the IS/OOS comparison using the parameters selected in Chapter 5. The main comparison is between two workflows: the standard workflow, optimized on the real historical IS period, and the synthetic workflow, optimized on synthetic IS paths. For MA Cross, an additional validation table is also shown to compare the real IS result with the synthetic-mean IS result using the same parameters.

In all tables, $\text{Diff} = \text{OOS} - \text{IS}$. This is the most important row, because it measures the change from backtested performance to OOS performance. For net profit, Sharpe, and average profit, a negative value means that OOS performance was worse than IS performance. For max DD, a negative value means that drawdown became larger OOS. The key comparison is between the standard workflow and the synthetic workflow, because both are tested on the same OOS data.

6.2 Moving-average crossover results

Sample	Net Profit	Max DD %	Sharpe	Avg Profit	Trades/Year
IS	44.56	-0.01	0.66	1.76	0.76
OOS	6.07	-0.18	0.02	-6.00	1.01
Diff	-38.49	-0.17	-0.64	-7.76	0.25

Table 6.1: MA Cross, standard workflow (fast=50, slow=200, no stop-loss).

Table 6.2 is not a separate optimization. It uses the same parameters selected by the standard workflow, but replaces the real IS result with the mean result across synthetic IS paths. The point is to check whether the synthetic paths give a similar IS estimate to the real historical IS period.

Sample	Net Profit	Max DD %	Sharpe	Avg Profit	Trades/Year
IS (synthetic mean)	37.44	-0.01	0.53	1.55	0.87
OOS (real)	6.07	-0.18	0.02	-6.00	1.01
Diff	-31.37	-0.17	-0.52	-7.54	0.14

Table 6.2: MA Cross, synthetic-path validation of the standard-workflow parameters (fast=50, slow=200, no stop-loss).

Sample	Net Profit	Max DD %	Sharpe	Avg Profit	Trades/Year
IS (synthetic mean)	37.45	-0.01	0.53	1.34	0.99
OOS (real)	-20.49	-0.21	-0.06	-8.96	1.01
Diff	-57.94	-0.20	-0.59	-10.30	0.02

Table 6.3: MA Cross, synthetic workflow (fast=35, slow=200, no stop-loss).

For MA Cross, the synthetic workflow does not improve the IS/OOS gap. Net profit deteriorates more than in the standard workflow (-57.94 versus -38.49), and OOS net profit becomes negative. The Sharpe gap is slightly smaller for the synthetic workflow (-0.59 versus -0.64), but the OOS Sharpe is still worse. For this strategy, the synthetic workflow does not improve the final OOS result.

6.3 Standard-deviation mean-reversion results

Sample	Net Profit	Max DD %	Sharpe	Avg Profit	Trades/Year
IS	33.07	-0.01	0.32	0.01	127.41
OOS	207.59	-0.16	0.41	0.21	122.40
Diff	174.52	-0.15	0.10	0.20	-5.01

Table 6.4: Std MRev, standard workflow (std_len=20, upper=1.0, lower=-0.35, slpct=2.5%).

Sample	Net Profit	Max DD %	Sharpe	Avg Profit	Trades/Year
IS (synthetic mean)	640.31	-0.14	0.70	0.39	84.92
OOS (real)	1842.71	-0.93	0.56	2.63	74.53
Diff	1202.40	-0.79	-0.14	2.24	-10.39

Table 6.5: Std MRev, synthetic workflow (std_len=150, upper=3.0, lower=-0.30, slpct=2.0%).

The dollar metrics need one important caveat. The tests use a fixed size of one lot. Since AAPL trades at higher prices in the OOS period, the same one-lot trade naturally creates larger dollar gains

and losses. This affects net profit and average profit, so those values are not enough by themselves to judge generalization. Sharpe and max DD percentage are more useful here because they are less dependent on the price level. For this reason, the Std MRev comparison is based mainly on Sharpe and max DD rather than dollar profit.

On Sharpe, the standard workflow shows no deterioration: the gap is +0.10. The synthetic workflow shows a deterioration of -0.14 . For Std MRev, the historically optimized parameters therefore generalize at least as well as the synthetically optimized parameters.

6.4 Summary

Strategy	Workflow	Net Profit Diff	Sharpe Diff
MA Cross	Standard	-38.49	-0.64
MA Cross	Synthetic	-57.94	-0.59
Std MRev	Standard	174.52	0.10
Std MRev	Synthetic	1202.40	-0.14

Table 6.6: IS-OOS gap comparison between the standard and synthetic workflows, same OOS period.

Across the two strategies, the synthetic workflow does not produce a consistently smaller IS/OOS gap.

Chapter 7

Conclusion

7.1 Summary of findings

This thesis compared two ways of optimizing rule-based trading strategies. The standard approach optimizes directly on the historical IS period. The alternative approach optimizes on multiple synthetic paths generated from that same IS period. Both approaches are then tested on the same real OOS period.

The results answer two different questions. First, whether the synthetic paths are a reasonable approximation of the real IS market. Second, whether optimizing on those paths reduces the IS/OOS performance gap.

7.2 Synthetic markets as realistic path approximators

On the first question, the results are positive. When the same benchmark parameters were evaluated on the synthetic IS paths for MA Cross, the mean synthetic performance remained close to the real IS performance. I did not repeat this same-parameter validation for Std MRev, because Figure 4.3 already shows that the threshold-constrained paths remain close to the original market path. The generator therefore seems to produce paths that remain connected to the original market, rather than unrelated random walks with the same starting price.

7.3 Does synthetic-market optimization reduce overfitting?

On the second question, the result is weaker. In these experiments, synthetic-path optimization does not show a clear reduction in overfitting. Across the two strategies:

- For MA Cross, the synthetic workflow produced a larger net profit gap than the standard workflow (-57.94 versus -38.49). The Sharpe gap was slightly smaller, but the OOS Sharpe was still worse.
- For Std MRev, the standard workflow had a positive Sharpe gap, while the synthetic workflow had a negative Sharpe gap of -0.14 .

The expected pattern would be a consistently smaller IS/OOS gap under the synthetic workflow. That pattern does not appear. With two strategies and one asset, this is not enough to reject the idea, but it is enough to say that the evidence in this thesis does not support it.

The two findings should be kept separate. The synthetic paths behave reasonably well as market approximations. However, this does not automatically mean that optimizing on them improves OOS performance.

7.4 Limitations

Several limitations matter for the interpretation of these results.

All experiments were run on a single asset, AAPL, over a single historical window and a single 70/30 IS/OOS split. Strategy performance and IS/OOS gaps can change across assets and market regimes.

All experiments used a fixed position size of one lot rather than a fractional or equity-scaled sizing rule. Because AAPL trades at higher prices in the OOS period, dollar metrics such as net profit and average trade are affected by the price level. This is why Sharpe and max DD percentage were given more weight in the discussion.

Parameter selection at every stage, in both the historical and synthetic workflows, relied on visual inspection of grid plots for robust regions rather than automatic maximization. This was intentional, but it remains partly discretionary.

The synthetic generator itself has one free parameter, the threshold multiplier k (the number of rolling standard deviations allowed around the original price), which was fixed at $k = 10$ throughout and not optimized. The volatility window was likewise fixed at 30 bars. Both choices affect the generated paths, so the results are conditional on these settings.

In theory, synthetic optimization could allow a larger part of the original historical series to be kept for real-data testing, because parameters are selected on generated paths rather than directly on the real path. This thesis does not use that design. A 70/30 split is kept so that the synthetic workflow can be compared directly with the standard IS/OOS benchmark. Also, any data used to calibrate or anchor the synthetic generator should not be treated as fully OOS.

The optimizer is a plain grid search and is not parallelized. This constrained how fine and how wide

the parameter grids could be. In the synthetic workflow, each parameter combination is tested on every synthetic path, so using 50 paths means running 50 backtests for one grid point.

7.5 Future work

A natural extension is to repeat the comparison across more assets, more historical windows, and more strategies. This would show whether the pattern observed here is general or specific to this dataset. Using fixed-fractional sizing would also remove the price-level issue in dollar metrics.

On the synthetic-generation side, it would be useful to treat the threshold multiplier and the volatility window as parameters to test, instead of fixed constants. The generator should also be compared with other synthetic-data methods, such as block bootstrapping of historical returns.

Finally, the optimizer should be parallelized before this method is scaled further. At the moment, a synthetic optimization with 50 paths runs 50 backtests for each parameter combination. Given that its OOS deterioration was not more predictable than the benchmark workflow in these experiments, the extra computation is not clearly justified yet. Parallelization would make wider grids and more synthetic paths practical, reducing noise without making the runtime excessive.

Code Availability

All code used for this thesis was developed independently by the author and is publicly available at: https://github.com/JacopoMichelacci/100_OOS_BSc_Thesis.

The thesis focuses on the research design, optimization procedure, synthetic market generation, and empirical results. For this reason, implementation details are discussed only when they are necessary to understand the methodology. The full source code is provided in the repository for reproducibility and further inspection.

Bibliography

- White, H. (2000). A reality check for data snooping. *Econometrica*, 68(5), 1097–1126.
- Bailey, D. H., Borwein, J. M., López de Prado, M., and Zhu, Q. J. (2017). The probability of backtest overfitting. *Journal of Computational Finance*, 20(4), 39–69.
- Sullivan, R., Timmermann, A., and White, H. (1999). Data-snooping, technical trading rule performance, and the bootstrap. *The Journal of Finance*, 54(5), 1647–1691.
- Politis, D. N., and Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association*, 89(428), 1303–1313.
- Brock, W., Lakonishok, J., and LeBaron, B. (1992). Simple technical trading rules and the stochastic properties of stock returns. *The Journal of Finance*, 47(5), 1731–1764.